

Understand the project. Explain the decisions.

A beginner's guide to a secure telecom assistant

Code walkthrough, security boundaries, demo preparation and interview answers.

THE IDEA TO REMEMBER

Language helps select the task. Trusted code controls access. Tools supply evidence. Secrets stay outside the model.

READING MAP

Pages 2-6: the application, architecture, request path, agent and tools.

Pages 7-9: identity, authorization, secrets and confirmation.

Pages 10-14: RAG, SQL, memory, observability and testing.

Pages 15-19: Docker, code tour, interview answers, pitches and GitHub safety.

Prepared for this repository review | 28 September 2026

Synthetic data only. Read README.md for current run commands and verification notes.

The project in plain English

Simply

Switchboard is a telecom support assistant. A telecom company has customers, installed devices, work orders, network alarms and technical procedures. An employee asks a question in ordinary language. The app finds relevant evidence and checks what that employee may access before returning an answer.

The main example

Maya is assigned to repair the subscriber at 14 Elm St. She can request a Wi-Fi password reveal while on shift using a managed device. Raj works in customer support: he may send a password reset, but may not read the existing password. The same question produces different permitted outcomes because of code-enforced policy.

What runs

One FastAPI application, one orchestrator, a shared tool registry and two SQLite database files. Mock operational APIs run in process. Procedure retrieval uses SQLite full-text search and small local hash vectors. Optional cloud adapters are separate choices, not prerequisites.

Why this exists

The project demonstrates that a useful assistant needs reliable access boundaries, evidence and tests. A clever prompt alone cannot protect customer information. A small explicit workflow is easier for an interviewer to review than an unexplained collection of frameworks.

Say in an interview

"I built a telecom assistant whose tools enforce authorization. The model can help understand a request, but trusted code controls what runs and what data is returned."

Keep the claim accurate

All operational data is synthetic. Persona selection is demo login. The reveal card simulates recent authentication, not real MFA. This is a reviewable prototype, not a production credential service.

Architecture: one path to understand

The path

User -> FastAPI -> Orchestrator -> Guarded tools -> Data or procedures -> Grounded response.
Authentication happens at the API boundary. Authorization happens at the tool and data boundaries.
Audit and tracing surround those operations.

FastAPI

An API is a set of endpoints another program can call. The browser posts a message to /v1/chat.
FastAPI and Pydantic reject malformed input, then the gateway verifies the token and constructs the current user principal.

Orchestrator

An orchestrator coordinates work. It rewrites abbreviations, resolves a previous customer reference, chooses an intent, builds a small dependency plan, executes ready steps, and formats the evidence. It does not run a hidden fleet of agents.

Tools and data

A tool is an ordinary Python function with a name, input schema and declared action/resource. Tools read subscriber data, request a reveal reference, query device status, retrieve procedures or execute safe SQL. Tools are not allowed to skip the shared access checks.

Why these boundaries

The browser handles interaction, the gateway handles identity, the orchestrator handles coordination, and tools handle business operations. Keeping these responsibilities separate makes failures and security checks easier to locate.

Optional pieces

Permit adds an external policy decision. Gemini adds language understanding and SQL generation. Neo4j can answer topology questions. Langfuse displays exported traces. The default offline demo needs none of them.

Interview example

"I can show the complete request path in three files: gateway.py, orchestrator.py and tools.py. Policy and retrieval are separate modules because they have different responsibilities."

Walk through one request

1. Identify and validate

Maya asks: "What is the Wi-Fi password at 14 Elm St?" The API validates nonempty text and length, verifies the JWT, and reads Maya's current role. An old token does not keep a revoked role alive.

2. Rewrite and route

The app recognizes the address and password intent. With Gemini enabled, a constrained model classifier can choose one of six intents. With no key, deterministic patterns choose it. Subject identifiers are extracted or recovered by code, not invented by the model.

3. Build the plan

The first step finds the subscriber. Later steps depend on the subscriber's device identifier: create a credential reveal grant and search relevant procedures. A failed or denied prerequisite blocks dependent steps.

4. Authorize the tools

The registry validates arguments and checks actions such as `read_profile` or `reveal_credential`. Finding a subscriber also checks the specific subscriber. Maya must have a current assignment relationship to the target device and satisfy the context requirements.

5. Return a reference

The chat response includes a short-lived reveal reference and a UI card. It contains no password. The card calls a separate redemption endpoint, which consumes the reference and checks fresh authorization before returning the credential directly to the UI.

6. Record useful evidence

The app records which tools ran, their allow/deny decisions and a trace ID. It remembers a subject reference for follow-up questions. It does not store the password in conversation state.

Interview example

"I check permission twice because access can change between requesting a card and redeeming it. That prevents a closed work order from leaving an old reveal card authorized."

The agent and the actual LLM role

Simply

An agent-like workflow uses language understanding to choose useful operations. It does not have to let a model execute arbitrary code. This project uses one constrained router and explicit Python plans.

In this project

llm.py optionally calls Gemini through google-genai. The classifier may return exactly one intent: wifi_password, psk_reset, outage, fix_hold, data_query or procedure. Extra fields and unknown intents are rejected. Failure falls back to deterministic routing.

Useful example

"The connection at 14 Elm St has gone completely silent" is an outage paraphrase that need not match the offline keyword rules. Gemini can classify it, after which the same Python outage plan reads topology, device status and procedures.

The second model use

For a structured question not covered by a SQL template, Gemini can propose a SELECT over the canonical schema. That SQL still passes AST validation, exact row scoping and read-only execution. A generated query is a proposal, not a trusted command.

What the model cannot do

It cannot choose the authenticated user, change a role, bypass a denied tool, mark a reset confirmed, or read a Wi-Fi password. Final operational answers are formatted from evidence rather than freely generated claims.

How to demonstrate it

Set GOOGLE_API_KEY in the same PowerShell session before starting the host app. Inspect routing_source in response details and the gen_ai.generate trace span. Offline tests use mocked model responses: they test the boundary, not live Gemini accuracy.

Interview example

"I constrained the model to intent classification because the business workflows are known. That gives me language flexibility while keeping the execution and security paths reviewable."

Tools, plans and failure handling

Simply

A tool is a named operation the assistant can request. A plan says which operations must happen and which earlier results they need. For example, device status needs a device identifier from subscriber lookup.

In this project

tools.py has one registry shared by the main orchestrator and optional adapters. Each tool declares its permission action, resource type, identifier argument and read/write properties. Pydantic validates required fields, types, extra fields and identifier patterns before execution.

Execution

The orchestrator executes independent ready steps concurrently. It caps tool calls and retry iterations and stops scheduling after a deadline. Dependent steps can be revisited after a transient prerequisite succeeds. HTTP clients have their own timeouts.

Failure behavior

Unknown tools and invalid inputs return safe error codes. Unexpected upstream errors become a generic unavailable message; private exception details are not echoed to the user. Read operations may be retried. HTTP writes are not retried automatically unless an idempotency key makes the retry safe.

Why no automatic outage writes

A question asking what is wrong should not silently book a technician. The default outage plan gathers evidence only. Reset and reboot require an explicit, action-bound confirmation reference.

Important limitation

A Python thread already executing an upstream call cannot be forcibly stopped by the scheduling deadline. Do not describe the deadline as a hard wall-clock cancellation guarantee.

Interview example

"Every tool call crosses the same validation and authorization boundary. My tests cover malformed arguments, denied access and a broken upstream service, not just the happy path."

Authentication, authorization and RBAC

Authentication: who are you?

A JWT is a signed token containing identity claims. `auth.py` verifies its signature, issuer and intended audience. The gateway then reads the current directory role and tenant. A token that was valid before a role change does not preserve that old role.

Authorization: may you do this?

Authorization checks an action against a resource for the current principal. `read_profile` on `subscriber:S-88123` is different from `reveal_credential` on `cpe_device:CPE-5521`. Being signed in does not mean every action is allowed.

RBAC

Role-based access control gives capabilities to job roles. Customer support may trigger a password reset; a NOC engineer may read topology. Broad role grants are useful, but they are not enough for sensitive field access.

Relationships

A technician is assigned to a work order. That work order serves a subscriber. The subscriber owns a device. Following these relationships derives device access without manually creating a direct permission for each device.

Why both

A role answers what kind of work someone may do. A relationship answers which specific customer or device the work concerns. Maya's technician role does not allow her to read every customer's credential.

Demo versus production

The persona menu mints tokens for synthetic users. It proves the authorization flow but is deliberately not a real identity provider. Real deployment would use enterprise login and verified step-up authentication.

Interview example

"Raj is authenticated, but credential reveal is unauthorized. I distinguish those cases in code and test them separately."

Context, expiry and the secret boundary

ABAC in simple terms

Attribute-based access control uses facts about the request, user or resource. For credential reveal, the app checks `on_shift` is true, `device_trust` equals the managed enum, contractor certification when relevant, and a finite recent-authentication age from zero to less than 300 seconds.

Live facts

Shift, device trust and certification are read from the app-owned `current-facts` table on each check. The gateway computes authentication age at redemption. Permit receives live attributes rather than relying only on a previously synchronized profile.

Assignment or break glass

The required relationship is a valid assignment path or an active break-glass link. Two distinct supervisors can approve a 15-minute emergency link. Supervisors still satisfy the contextual checks; there is no unrestricted supervisor credential grant.

Expiry and revocation

Local tuple queries exclude expired rows immediately, including a tuple one second past expiry. Closing an assignment deletes the local replicated tuple in the transaction. A durable outbox retries external synchronization. Remote delay cannot turn that local denial into an allow.

Secret by reference

The chat receives a random reference, with only its hash stored. It expires after 60 seconds, belongs to one user, and can be redeemed once. The separate response has no-store caching headers. Secret endpoint responses never enter recorded HTTP cassettes.

Why this matters

Preventing a password from entering model context is stronger than hoping a model or a redaction regex will hide it later. Scrubbing remains a second layer.

Interview example

"I minimize exposure architecturally: the model receives a reveal reference, and only an authorized UI redemption can fetch the credential."

Human confirmation is data, not a word

The problem

If the app treats the text "confirmed" as approval, any user or model can add that word. The request is not reliably bound to the exact action the user reviewed.

The implementation

A reset or reboot first creates a random confirmation reference. Its hash is stored with the actor, action, exact validated arguments and expiry. The UI displays a confirmation button. Pressing it sends only the reference to `/v1/confirm/{action}`.

At redemption

The database atomically marks the matching unexpired reference used and returns the stored arguments. The gateway establishes a narrowly scoped approval context, then calls the guarded tool again. Current authorization is checked before the write.

What fails

Another user cannot redeem it. The same reference cannot approve a different action. Expired or already-used references fail. Extra caller-supplied arguments are rejected. A direct tool argument named confirm is not accepted from clients.

Why this is understandable

The reference is a receipt for one proposed operation. It does not grant general permission. It lets the server prove which operation was approved without trusting a natural-language statement.

Reset versus reveal

A reset sends a new credential to the account holder through a mock operational service; it does not return the credential to support chat. A reveal displays an existing credential only through the separate authorized card.

Interview example

"I fixed a confirmation bypass by replacing a text flag with a single-use token bound to the user and exact arguments. A regression test proves that typing confirmed does not execute the write."

RAG: retrieve evidence before answering

Simply

Retrieval-augmented generation means finding relevant source material before answering. Here the procedure answer is deliberately extractive: the app returns documented sections and citations rather than asking a model to invent operational steps.

Ingestion

rag.py reads document metadata and text, splits headings into parent sections, and creates smaller searchable child chunks. Numbered instructions stay together where possible. A detected injected instruction quarantines the whole document. Reingesting a document replaces old chunks and search entries rather than duplicating them.

Retrieval

SQL filters out quarantined chunks and disallowed classifications, applies device-model filters, and selects the latest revision in a family. SQLite FTS5 gives lexical matches; local hash vectors provide another rank. Reciprocal rank fusion combines ranks. With no lexical evidence, the app returns no procedure.

Grounding and citations

The response includes the surrounding parent section, revision and a reference such as DOC-MOP-ONT-r2#Procedure/p2. In these synthetic documents p2 is a section ordinal, not proof of an original PDF page number.

Why this approach

Telecom maintenance needs exact preconditions, steps and rollback. Keeping a whole section avoids returning an isolated instruction without its surrounding constraints. Classification filtering prevents the assistant from exposing a restricted manual.

Limitations

Hash vectors are cheap deterministic lexical features, not pretrained semantic embeddings. Injection detection is heuristic. A larger real-language retrieval dataset is needed before claiming production recall.

Interview example

"For ONT replacement I return the current permitted procedure and its rollback section, with citations. I do not let the model fabricate repair steps."

SQL: flexible questions, narrow access

Simply

SQL asks structured questions of tables: counts, lists and relationships. Natural-language-to-SQL is useful, but generated SQL must be treated as untrusted input.

The data model

The synthetic legacy database has awkward telecom names such as `sub_mstr` and `cpe_inv`. `ontology.yaml` maps them into canonical views like `canon_customer` and `canon_resource`. The views expose useful business names and omit classified PII and secret columns.

Generating a query

Known questions use approved templates. Other questions may use Gemini with only the safe schema. `sqlglot` parses the result into a syntax tree and rejects multiple statements, unsupported relations, forbidden columns/functions, and invalid limit or offset expressions.

The critical row boundary

The application computes the exact subscriber IDs the user may access and builds views scoped to those IDs. Sharing a postcode never grants access. Maya sees her one assigned customer in 02139; a broader permitted internal support role can see the broader count.

Execution protection

The source database is attached read-only. Query-only mode is enabled. The query has a progress timeout and returns at most 200 rows. Generated SQL cannot undo a scope that is already built into the view.

Why validation is layered

An AST check limits the query shape; safe views limit data exposure; read-only execution limits effects; resource budgets limit cost. Each protects against a different kind of mistake.

Interview example

"I fixed a real row-scoping bug: technician access was based on ZIP codes, which exposed neighbours. It now uses exact authorized customer IDs, with a regression test comparing one versus 24 rows."

Memory: context is not permission

Simply

Memory lets a follow-up such as "same customer" refer to an earlier subject. It should reduce repetition without becoming a hidden source of stale access.

Short-term memory

Session state is keyed by user and session, stores the last subject reference, and retains at most 20 turn summaries. A user cannot overwrite another user's session by choosing the same session name. The state contains no passwords or complete tool payloads.

How a follow-up works

After Maya asks about S-88123, the next same-session question can resolve that identifier from state. The next tool still checks Maya's current permissions. If her work order closed, the remembered identifier cannot make the new request succeed.

Optional long-term memory

`SB_LONG_TERM_MEMORY=1` enables cross-session episodes. Recall first queries only rows belonging to the current user and not past expiry. It then reauthorizes the referenced subscriber before returning a summary. This preserves a second data-layer scope even when Permit approves access.

Why opt-in

A simple interview demo needs only follow-up context. Cross-session retention adds privacy and lifecycle questions, so it is optional. Physical cleanup can occur later because expired records are already excluded from reads.

What to avoid claiming

The app does not learn a permanent model of a person or train Gemini on conversations. Stored references are application state. Short-term here means session-purpose data, not a guaranteed automatic physical deletion deadline.

Interview example

"I separate remembering an identifier from being authorized to use it. Recall is user-scoped, and every subsequent tool call is authorized again."

Logging, tracing and audit

Three different questions

A log records an operational event. A trace groups the steps of one request and their timing/context. An audit record explains a security decision. They overlap, but they serve different readers and retention needs.

In this project

logging_config.py emits JSON metadata events for authorization and tool completion. telemetry.py records a local span buffer and optionally exports OpenTelemetry spans to Langfuse. audit.py appends policy decisions linked by hashes.

Trace IDs

A request and its tool spans share a trace identifier. With OTel enabled, the application adopts the actual OTel trace ID so a developer can connect the API response to the cloud trace. Tool names, statuses and allow/deny reasons explain the path.

Data minimization

The password flow excludes credentials from chat and spans. Raw question text is not exported as a span attribute; a scrubbed local demo buffer supports debugging. Logs do not need tokens or full tool payloads to explain a failure.

Audit limits

SQLite triggers reject normal updates/deletes and hashes reveal ordinary chain tampering. Someone controlling the database can rewrite the entire store. A production design needs a separate protected audit destination.

Why Langfuse is optional

Local metadata is enough to understand the core demo. Langfuse becomes useful for inspecting many runs and model calls. Exporting metadata to a cloud service is an explicit configuration choice.

Interview example

"When an action is denied, I can show the policy reason and tool trajectory using one trace ID, without logging the Wi-Fi password."

Tests and evaluations

Simply

Tests prove specific expected behavior. Evaluations measure useful qualities across a set of examples. A passing unit test is not the same as representative model accuracy.

What is tested

Allowed and denied reveal, expired relationships, live context, confirmation reuse and expiry, role revocation, exact SQL scope, malformed inputs, tool failures, poisoned documents, reingestion, memory boundaries and protocol authorization. Model routing tests supply fake responses to check valid choices and safe fallback.

Isolation

conftest.py clears inherited cloud settings and copies a clean synthetic database for each test. Tests do not reuse your live .sbd data or depend on previously granted break-glass access. This makes failures reproducible and avoids paid cloud calls.

The evaluation gate

Small gold sets compare SQL execution results, document hit@3, classification F1 and a deterministic refusal rubric. Recorded trace fixtures check that important tool trajectories and policy decisions do not silently change.

What the numbers mean

The reviewed installed environment passed 115 tests before the final publication-hygiene pass. These checks demonstrate the implemented boundaries on synthetic data. They do not establish quality on every paraphrase or every real telecom dataset.

Optional judge

DeepEval/GEval can be enabled separately for an LLM judge. This repo does not claim a nightly calibrated judge pipeline. Judges cost API calls and need human-labelled examples and calibration to be meaningful.

Interview example

"I test a denied user, an expired grant and a failing service alongside the successful flow. My offline evaluation gate is repeatable, and I distinguish it from live model quality testing."

Docker and running the project

Simply

An image packages the app and its dependencies. A container runs that image. Docker Compose describes how to start it and where to keep its data. Docker is not an API-key vault and does not make public credentials safe.

The default

compose.yml starts one application with local adapters, no cloud credentials and synthetic persona login. Port 8000 is bound to localhost. The image runs as UID 10001, with a writable /data directory and a named volume for persistence.

Host start

From the project folder, create a virtual environment and install .[dev]. Run: .venv/Scripts/python -m switchboard.cli --offline --demo serve gateway. Open <http://127.0.0.1:8000/>. Existing configured cloud users can run start.ps1 instead.

Container start

Run docker compose up --build -d. Use docker compose ps to inspect it and docker compose down to stop it. Stop any host app already using port 8000 first. The named volume survives a normal down command.

Cloud override

compose.cloud.yml adds optional packages and reads runtime environment variables from .env. The separate Permit container remains on port 7766; the app container reaches it through host.docker.internal. Shell-only host keys do not automatically appear inside a container.

Verification

A clean image was built and started on isolated localhost port 18080. HTTP checks exercised health, allowed/denied reveal, single-use behavior, confirmation, SQL scoping, read-only outage diagnosis and RAG. No cloud keys were passed to that smoke container.

Interview example

"A reviewer can run one container without opening cloud accounts. The optional integrations use runtime settings and do not change the core workflow."

Repository tour and JD mapping

Read these first

gateway.py: API and identity boundary. orchestrator.py: routing and coordination. tools.py: schemas and guarded execution. policy.py: permission decisions. tests/test_regressions.py: concrete security mistakes prevented by the code. Source files live under src/switchboard/.

Then follow the data

core.py manages SQLite connections; datagen.py creates synthetic records; legacy.py emulates operational APIs. text2sql.py and ontology.py expose safe structured access. rag.py retrieves procedures. memory.py implements optional episode recall.

Security and operations

auth.py verifies tokens; idg.py manages relationship changes and the outbox; reveal.py and confirmations.py manage single-use references. telemetry.py, logging_config.py and audit.py explain what ran and why.

JD: direct evidence

Agent orchestration and tools: one router and typed plans. RAG/SQL: current cited procedures and exact data scopes. Identity/context: JWT, roles, assignment and fresh attributes. Production engineering: validation, errors, tests, JSON events and a nonroot Docker image.

JD: optional evidence

Gemini/Vertex: google-genai adapter. Permit: external policy decision with local safeguards. Langfuse/OTel: trace export. DeepEval: optional judge adapter. MCP/A2A/ADK: extension examples outside the default path. Neo4j: optional topology queries.

What was deliberately excluded

No Kubernetes/GKE deployment, no default multi-agent fleet, and no mandatory graph or vector server. Installing technologies just to match a job description would make this project harder to explain without improving its core demonstration.

Interview example

"I chose a small architecture because the important engineering problem is controlled access to tools and evidence. I can explain where a managed database or cloud runtime would fit later."

Likely interview questions

Why not let the LLM call any tool?

The workflows are known and some actions expose sensitive data or create side effects. A constrained intent plus explicit plan is easier to validate, test and authorize. More autonomy would require a stronger evaluation and approval design.

Can prompt injection bypass authorization?

The injection detector is only a heuristic. The important boundary is that every tool is checked by code, queries are scoped, and credentials never enter model context. A model choosing the wrong intent still cannot grant itself access.

Why is a password reveal different from a reset?

Reveal reads an existing secret; reset requests a change delivered to the account holder. Support can help without seeing the secret. The two actions have separate permissions and UI flows.

What if Permit is down or stale?

Configured object checks require Permit to allow as well as the local guard. Remote errors deny. Locally expired or revoked relationships cannot be made valid by a stale remote allow. The outbox retries synchronization.

How do you prevent hallucinated repairs?

Retrieve permitted current procedures and return cited sections. If there is no evidence, say so. Device and topology claims come from tool results, not a model's general knowledge.

How do you know Gemini is actually used?

Check routing_source and the model span. Mocked unit tests verify integration behavior; a separate live paraphrase evaluation is needed to measure the configured model's language quality.

What would you change for production?

Real OIDC login and MFA, explicit tenant ownership, managed secrets, HTTPS, protected audit storage, distributed rate limits, realistic evaluations and load testing. Choose scaling infrastructure after measuring the workload.

Your 30-second and 3-5 minute explanations

30 seconds

"Switchboard is a telecom assistant with code-enforced tool authorization. It can find subscriber information, diagnose an outage, retrieve procedures and answer safe SQL questions. My main demo compares an assigned technician who can reveal a Wi-Fi password with support staff who cannot. The password bypasses the model through a single-use UI reference. I kept the core to one app, with regression tests, tracing and Docker; cloud integrations are optional."

3-5 minutes: problem and design

"Telecom support combines operational data, technical documents and sensitive credentials. I wanted a reviewer to understand the entire system, so I used one FastAPI app and one orchestrator. A request is validated and authenticated, rewritten when useful, routed to an intent and converted into an explicit tool plan."

Continue: security example

"The strongest example is Wi-Fi password access. Maya has an active assignment path from work order to subscriber to device. Reveal also needs shift, managed-device and recent-authentication checks. Raj may reset a password but cannot see it. Successful chat does not return the credential: it returns a short-lived reference. Redemption is single-use and rechecks policy, so changing access after the original request still matters."

Continue: evidence and LLM

"Gemini can understand paraphrases and propose SQL for unsupported questions. It cannot choose identity or bypass tool checks. SQL runs only over safe views scoped to exact customer IDs. Procedure answers use current permitted document sections with citations. Memory remembers a subject, but the next request is authorized again."

Continue: quality and trade-offs

"I added regression tests for invalid inputs, expired grants, role changes, failed tools and incorrect model outputs. Logs and traces explain decisions without credentials. The default Docker setup is one local app; Permit, Langfuse and Neo4j are optional. I also removed automatic outage writes. The demo login and simulated recent-authentication flow are explicit prototype limits, and I can explain what production identity and deployment would require."

Delivery tip

Pause after each paragraph and show the relevant UI or file. At a natural pace, with a short demonstration and transitions, this outline supports a three-to-five-minute explanation. Do not memorize framework names without understanding the boundaries.

Before the interview and GitHub upload

Know these five answers

Where is identity established? Where is tool permission checked? Why does the model never see the password? How are SQL rows and documents scoped? Which parts are real integrations versus local simulation? If you can point to each in code, you understand the project's core.

Rehearse the demo

Start offline. Show Maya allowed, Raj denied, a closed work order denied, a confirmed reset, a cited ONT procedure, an exact scoped customer count and a same-session follow-up. Reopen the demo work order afterwards. Show one trace and one regression test.

Keep keys local

.env contains real configuration and is ignored. .env.example contains names with empty values. Code reads environment variables. A shell-only GOOGLE_API_KEY works with host startup in that session. Never paste real keys into source, screenshots, issue reports or the public README.

Manual GitHub upload

Manual uploads do not apply .gitignore automatically. Use the checked source archive, extract it, and upload only its contents. Include .github, .gitignore and .env.example. Do not upload your whole Downloads/RAG directory: it also contains unrelated files. Do not upload the archive itself as the only repository file.

Check before publishing

Run scripts/check_secrets.py for Git upload candidates. It searches common token patterns and exact values from the local .env without printing them. Review git status and git diff --cached. A scan reduces risk; it is not a proof that every possible secret format is absent.

History and deployment

This inspected local Git repository had no commits and no remote. That says nothing about earlier archives or another repository. If a real key was ever published or committed elsewhere, rotate it. Uploading Python source to GitHub does not run the FastAPI backend; GitHub Pages cannot host this server.

Final mindset

Explain the trade-offs honestly. A small working security boundary, clear evidence path and meaningful tests are stronger interview material than a long list of unverified production claims.